CSP-S 模拟试卷（含答案与解析）

考试时间：120 分钟　满分：100 分　生成时间：2026/5/6 16:57:41

姓名：_____　班级：_____　得分：_____

请在答题区域作答；单项选择题、阅读程序题和完善程序题均为单选。除特别说明外，程序均按 C++17 标准理解。

一、单项选择题（共 15 题，每题 2 分，共 30 分）

1. （2 分） 给定一个包含 n 个元素的数组 a，以下 C++ 代码片段执行后，变量 ans 的值是多少？

```
int ans = 0;
for (int i = 0; i < n; ++i) {
    for (int j = i + 1; j < n; ++j) {
        if (a[i] > a[j]) ans++;
    }
}
```

A. 数组 a 的逆序对数量

B. 数组 a 中大于 a[0] 的元素个数

C. 数组 a 的最小元素索引

D. 数组 a 的总元素个数

答案： A

解析：

该代码使用双重循环枚举所有满足 i < j 的下标对 (i, j)，当 a[i] > a[j] 时计数器 ans 自增。这恰好统计了数组中所有逆序对的数量，即满足 i < j 且 a[i] > a[j] 的数对个数。

2. （2 分） 考虑一个有向无环图（DAG）的拓扑排序。如果使用Kahn算法（基于入度的算法），初始时入度为0的顶点有多个，那么最终得到的拓扑序列是否唯一？

A. 一定唯一

B. 一定不唯一

C. 可能唯一，取决于图的结构

D. 可能唯一，取决于顶点的处理顺序

答案： C

解析：

拓扑序列的唯一性完全取决于图的结构，而非处理顺序。当图中存在至少两个入度为0的顶点时，不同的处理顺序会导致不同的拓扑序列，因此序列不唯一。但若图的结构保证每一步都只有一个入度为0的顶点（例如一条链），则拓扑序列唯一。因此，拓扑序列可能唯一也可能不唯一，具体取决于图的结构。选项C正确。选项D错误，因为处理顺序本身不能改变唯一性，唯一性由图的结构决定。

3. （2 分） 以下C++代码中，函数`f`的作用是什么？

```
int f(int n) {
    if (n <= 1) return 1;
    return f(n-1) + f(n-2);
}
```

A. 计算n的阶乘

B. 计算斐波那契数列的第n项

C. 计算2的n次幂

D. 计算n的平方

答案： B
解析：
函数f递归调用f(n-1)和f(n-2)，基本情况是n<=1时返回1。这定义了斐波那契数列的递推关系，但初始项与标准定义略有不同：标准斐波那契数列F0=0, F1=1；而这里f(0)=1, f(1)=1。因此它计算的是斐波那契数列的一个变体，其中第n项对应标准斐波那契数列的第n+1项。例如，f(0)=1 (对应F1), f(1)=1 (对应F2), f(2)=2 (对应F3), f(3)=3 (对应F4), f(4)=5 (对应F5)。选项B描述最准确，因为它捕捉到了核心递推关系和数列本质，其他选项的递推关系均不匹配。

4. （2 分） 在二叉树中，如果一个节点的左右子树均为空，该节点称为叶子节点。一棵高度为h（根节点高度为1）的满二叉树有多少个叶子节点？

A. h

B. 2h

C. 2^{h-1}

D. 2^h - 1

答案： C
解析：
满二叉树每一层节点数都是上一层的两倍。高度为h时，叶子节点全部在第h层，共有2^{h-1}个。

5. （2 分） 在完全二叉树中，若节点总数为n，则叶子节点数至少为多少？

A. ⌈n/2⌉

B. ⌊n/2⌋

C. ⌈(n+1)/2⌉

D. ⌊(n+1)/2⌋

答案： D
解析：
在完全二叉树中，叶子节点数最少的情况是最后一层只有最左侧有节点。设树高为h，则前h-1层节点总数为2^(h-1)-1，最后一层至少有一个节点。当n=2^(h-1)-1+1=2^(h-1)时，叶子节点数最少，为1+（前h-2层满节点的叶子节点数）。具体计算：当n为奇数时，叶子节点数=(n+1)/2；当n为偶数时，叶子节点数=n/2。统一公式为floor((n+1)/2)。例如n=4时，叶子最少为2，floor((4+1)/2)=2；n=3时，叶子最少为2，floor((3+1)/2)=2；n=2时，叶子最少为1，floor((2+1)/2)=1。因此正确答案为D。

6. （2 分） 以下 C++ 代码段执行后，输出结果是（ ）。

```
int a = 5, b = 10;
a = a ^ b;
b = a ^ b;
a = a ^ b;
cout << a << " " << b;
```

A. 5 10
B. 10 5
C. 15 5
D. 10 10

> **答案：** B
> **解析：**
> 使用异或交换：a=a^b=5^10=15；b=a^b=15^10=5；a=a^b=15^5=10。最终 a=10, b=5。

7. （2 分） 使用邻接矩阵存储有向图，该矩阵大小为 n×n，每个元素为 int 型。若进行深度优先遍历，其时间复杂度的最紧上界是?

A. O(n)
B. O(n log n)
C. O(n^2)
D. O(n^3)

> **答案：** C
> **解析：**
> 邻接矩阵中，查找一个顶点的所有邻接点需要扫描该行，复杂度 O(n)。DFS 访问每个顶点，对于每个顶点，检查其所有邻接点，因此总时间复杂度为 O(n^2)。这是最紧上界。

8. （2 分） 一个空栈依次执行 8 次入栈、1 次出栈后，栈中还剩几个元素?

A. 7
B. 5
C. 6
D. 8

> **答案：** A
> **解析：**
> 栈的入栈增加元素、出栈减少元素，剩余数量为 8 - 1 = 7。

9. （2 分） 在 C++17 中，运行以下程序，输出是什么?

```
#include <iostream>
int main() {
    int arr[] = {1, 2, 3, 4, 5};
    std::cout << (*(arr + 2)) << std::endl;
    return 0;
}
```

A. 1
B. 2
C. 3
D. 4

> **答案：** C
> **解析：**
> arr 是数组首地址，arr+2 指向第三个元素，*(arr+2) 等价于 arr[2]，值为 3。

10. （2 分） 以下C++表达式中，哪一个的值与其他三个不同？ 已知：`int a = 2, b = 3;`

A. `a + b`

B. `a * 2 + b - a`

C. `(a << 1) + b - a`

D. `a * b - a`

> **答案：** D
> **解析：**
> 计算各表达式：
> A. `a + b` = 2 + 3 = 5
> B. `a * 2 + b - a` = 2*2 + 3 - 2 = 4 + 3 - 2 = 5
> C. `(a << 1) + b - a` = (2左移1位即4) + 3 - 2 = 4 + 3 - 2 = 5
> D. `a * b - a` = 2*3 - 2 = 6 - 2 = 4
> 可见A、B、C的值均为5，而D的值为4。因此D的值与其他三个不同。

11. （2 分） 以下 C++ 代码的输出是什么？

```
#include <iostream>
using namespace std;
int main() {
    int a = 10, b = 3;
    cout << (a / b) << " " << (a % b);
    return 0;
}
```

A. 3.333 1

B. 3 1

C. 3 0

D. 4 -2

> **答案：** B
> **解析：**
> `a / b` 是整数除法，10 除以 3 的整数部分是 3。`a % b` 是取余数，10 除以 3 的余数是 1。因此输出为 "3 1"。

12. （2 分） 用 1,2,3,4 四个数字组成无重复数字的四位数，将所有这样的四位数从小到大排列，则第 16 个数是多少？

A. 3124

B. 3142

C. 3214

D. 3241

> **答案：** D
> **解析：**
> 全排列共24个。千位1开头第1-6个，千位2开头第7-12个，千位3开头第13-18个。第16个是千位3开头的第4个数。千位3时，百位1有2个(3124,3142)为第13-14个；百位2有2个(3214,3241)为第15-16个；故第16个是3241。

13. （2 分） 在C++中，表达式 1 + 2 << 3 的值是多少？

A. 9

B. 24

C. 25

D. 26

> **答案：** B
> **解析：**
> 加法的优先级高于左移，所以等价于 (1 + 2) << 3 = 3 << 3 = 3 * 8 = 24。注意左移是位运算，不是算术。

14. （2 分） 在 C++ 中，以下代码的输出是什么？

```
#include <iostream>
using namespace std;
int main() {
    int a[] = {1, 2, 3, 4, 5};
    int *p = a + 2;
    cout << *p++ << ' ' << *p;
    return 0;
}
```

A. 3 4
B. 3 3
C. 4 4
D. 4 5

> **答案：** A
> **解析：**
> `p` 初始指向 `a[2]`，即值为 3。表达式 `*p++` 先取 `p` 所指向的值（3），然后 `p` 自增指向下一个元素（`a[3]`，值为 4）。因此第一个输出为 3，此时 `p` 已指向 4，第二个输出为 `*p` 即 4。故输出 `3 4`。

15. （2 分） 以下C++代码的输出是什么？

```
#include <iostream>
#include <vector>
int main() {
    std::vector<int> v = {1, 2, 3, 4, 5};
    auto it = v.begin() + 2;
    v.insert(it, 6);
    std::cout << *it << " " << v[2];
}
```

A. 3 6
B. 6 3
C. 未定义行为
D. 3 3

> **答案：** C
> **解析：**
> 在vector中insert可能导致迭代器失效。it在执行insert后可能失效，解引用已失效的迭代器是未定义行为。因此输出不确定，可能崩溃。

二、阅读程序题（共 3 题，共 40 分）

16. （本题 14 分） 阅读以下程序，回答问题。

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n = 5;
    vector<int> a = {3, 1, 4, 1, 5};
    vector<int> b;

    for (int i = 0; i < n; ++i) {
        int cnt = 0;
        for (int j = 0; j < n; ++j) {
            if (a[j] < a[i]) {
                cnt++;
            }
        }
        b.push_back(cnt);
```

```
    }

    int max_val = 0;
    for (int i = 0; i < n; ++i) {
        if (b[i] > max_val) {
            max_val = b[i];
        }
    }

    cout << max_val << endl;

    int ans = 0;
    for (int i = 0; i < n; ++i) {
        if (b[i] == max_val) {
            ans = a[i];
        }
    }

    cout << ans << endl;
    return 0;
}
```

16.1 （3 分） 程序第一次输出的值是多少？

A. 2

B. 3

C. 4

D. 5

> **答案：** C

16.2 （3 分） 程序第二次输出的值是多少？

A. 1

B. 3

C. 4

D. 5

> **答案：** D

16.3 （3 分） 变量cnt在第一次外层循环（i=0）结束后，其值是多少？

A. 1

B. 2

C. 3

D. 4

> **答案：** B

16.4 （3 分） 变量max_val在整个程序执行过程中被赋值的次数是多少？（包括初始化）

A. 1

B. 2

C. 3

D. 4

> **答案：** D

16.5 （2 分） 程序最终变量ans的值是多少？

A. 1

B. 3

C. 4

D. 5

答案： D

程序输出：

```
4
5
```

整题解析：

1. 数组a为{3,1,4,1,5}。对于每个a[i]，计算比它小的元素个数cnt并存入b。具体地：a[0]=3，比3小的元素有1和1，共2个；a[1]=1，比1小的元素没有，共0个；a[2]=4，比4小的元素有3,1,1，共3个；a[3]=1，共0个；a[4]=5，比5小的元素有3,1,4,1，共4个。因此b数组为{2,0,3,0,4}。程序第一次输出b中的最大值max_val，即4。
2. 由上一步分析，b={2,0,3,0,4}，max_val=4。程序第二次输出满足b[i]==max_val的a[i]。b中只有b[4]=4，此时a[4]=5，因此输出5。
3. 当i=0时，a[0]=3。内层循环j从0到4，检查a[j] < 3的条件：j=0时3<3不成立；j=1时1<3成立，cnt变为1；j=2时4<3不成立；j=3时1<3成立，cnt变为2；j=4时5<3不成立。因此内层循环结束后cnt=2。
4. 程序中max_val首先被初始化为0（第一次赋值）。随后在循环中，当i=0时，b[0]=2>0，max_val被赋值为2（第二次赋值）。当i=1时，b[1]=0不大于当前max_val（2），不赋值。当i=2时，b[2]=3>2，max_val被赋值为3（第三次赋值）。当i=3时，b[3]=0不大于3，不赋值。当i=4时，b[4]=4>3，max_val被赋值为4（第四次赋值）。因此赋值次数为4次（包括初始化），对应选项D。
5. 由前面的分析，max_val=4，且只有b[4]=4，因此ans被赋值为a[4]=5。最终输出ans为5。

17. （本题 13 分） 阅读以下程序，回答问题。

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n = 6;
    vector<int> a = {3, 1, 4, 1, 5, 9};
    vector<int> b(n, 0);

    for (int i = 0; i < n; ++i) {
        int cnt = 0;
        for (int j = 0; j < i; ++j) {
            if (a[j] < a[i]) {
                cnt++;
            }
        }
        b[i] = cnt;
    }

    int total = 0;
    for (int i = 0; i < n; ++i) {
        total += b[i];
    }

    cout << "总和: " << total << endl;
    for (int i = 0; i < n; ++i) {
        cout << b[i] << " ";
```

```
    }
    cout << endl;

    return 0;
}
```

17.1 （3 分） 程序运行后，第一行输出（"总和: "之后）的数值是多少？

A. 8
B. 9
C. 10
D. 11

> **答案：** D

17.2 （3 分） 程序输出的第二行，即 b 数组的内容，是什么？

A. 0 0 2 0 4 5
B. 0 1 2 0 4 5
C. 0 0 1 0 3 5
D. 0 0 2 1 4 5

> **答案：** A

17.3 （3 分） 在程序执行过程中，当 i=3 时，内层循环 j 从 0 到 2 共执行了多少次比较？

A. 2
B. 3
C. 4
D. 5

> **答案：** B

17.4 （2 分） 如果将程序中的 a 数组改为 {5, 4, 3, 2, 1, 0}，其他代码不变，那么输出的 total 值是多少？

A. 0
B. 1
C. 5
D. 15

> **答案：** A

17.5 （2 分） 程序的时间复杂度是多少？

A. O(n)
B. O(n log n)
C. O(n^2)
D. O(n^3)

> **答案：** C

程序输出：

```
总和: 11
0 0 2 0 4 5
```

整题解析：

1. 数组 a 为 {3,1,4,1,5,9}。程序计算 b[i] 为 a[i] 之前比 a[i] 小的元素个数。i=0时，前面无元素，b[0]=0；i=1时，a[0]=3>1，b[1]=0；i=2时，a[0]=3<4, a[1]=1<4，b[2]=2；i=3时，a[0]=3>1, a[1]=1不小于1, a[2]=4>1，b[3]=0；i=4时，a[0]=3<5, a[1]=1<5, a[2]=4<5, a[3]=1<5，b[4]=4；i=5时，所有前面元素都小于9，b[5]=5。总和 total = 0+0+2+0+4+5 = 11。因此输出 11，选项 D 正确。
2. 如上分析，b[0]=0, b[1]=0, b[2]=2, b[3]=0, b[4]=4, b[5]=5。所以输出为 "0 0 2 0 4 5"，对应选项 A。
3. i=3 时，内层循环 for (int j = 0; j < i; ++j) 中 i=3，所以 j 取值 0, 1, 2，共执行 3 次循环体，即进行 3 次比较。
4. 新数组 a 为降序序列。对于每个 i，b[i] 计算的是 a[i] 之前比 a[i] 小的元素个数。由于数组严格降序，每个 a[i] 之前的所有元素都大于 a[i]，没有比 a[i] 小的元素，因此所有 b[i] 均为 0。总和 total = 0。
5. 程序包含两层嵌套循环：外层循环 i 从 0 到 n-1，内层循环 j 从 0 到 i-1。在最坏情况下，内层循环执行次数为 0 + 1 + 2 + ... + (n-1) = n(n-1)/2，因此时间复杂度为 O(n^2)。

18. （本题 13 分） 阅读以下程序，回答第1-5题。

```
#include <iostream>
#include <stack>
#include <queue>
using namespace std;
int main() {
    int a[] = {5, 3, 8, 1, 6};
    stack<int> s;
    queue<int> q;
    for (int i = 0; i < 5; ++i) {
        if (a[i] % 2 == 1) {
            s.push(a[i]);
        } else {
            q.push(a[i]);
        }
    }
    cout << "S:";
    while (!s.empty()) {
        cout << " " << s.top();
        s.pop();
    }
    cout << endl;
    cout << "Q:";
    while (!q.empty()) {
        cout << " " << q.front();
        q.pop();
    }
    cout << endl;
    return 0;
}
```

18.1 （3 分） 程序执行后，第一行输出的"S:"之后的序列是什么？

A. 5 3 1

B. 1 3 5

C. 5 3 8 1 6

D. 6 1 8 3 5

> **答案：** B

18.2 （3 分） 程序执行后，第二行输出的"Q:"之后的序列是什么？

A. 8 6

B. 6 8

C. 5 3 1 8 6

D. 8 6 1 3 5

> **答案：** A

18.3 （3 分） 程序执行过程中，栈 s 的最大大小是多少？

A. 1

B. 2

C. 3

D. 4

> **答案：** C

18.4 （2 分） 程序执行过程中，队列 q 的最大大小是多少？

A. 1

B. 2

C. 3

D. 4

> **答案：** B

18.5 （2 分） 程序总共执行了多少次入栈操作？

A. 2

B. 3

C. 4

D. 5

> **答案：** B

程序输出：

```
S: 1 3 5
Q: 8 6
```

整题解析：

1. 数组 a 为 {5, 3, 8, 1, 6}。遍历数组，奇数入栈 s：5 是奇数入栈，3 是奇数入栈，8 是偶数入队 q，1 是奇数入栈，6 是偶数入队 q。栈 s 最终包含元素（从栈底到栈顶）：5, 3, 1。输出时从栈顶开始弹出，顺序为 1, 3, 5。代码中输出时是在弹出前输出 top，所以输出顺序就是 1, 3, 5。因此第一行输出的序列是 1 3 5，对应选项 B。
2. 数组 a 为 {5, 3, 8, 1, 6}。偶数入队 q：8 是偶数入队，6 是偶数入队。队列 q 包含元素 8 和 6，队首是 8，队尾是 6。输出时从队首开始弹出，顺序为 8, 6。因此输出为 "8 6"，对应选项 A。
3. 遍历数组 a：5 入栈 s，大小为 1；3 入栈 s，大小为 2；8 入队 q，栈大小不变；1 入栈 s，大小为 3；6 入队 q，栈大小不变。之后出栈时大小减小。因此栈 s 的最大大小是 3。
4. 遍历数组 a：5 和 3 入栈，队列大小为 0；8 入队，大小为 1；1 入栈，队列大小不变；6 入队，大小为 2。之后出队时大小减小。因此队列 q 的最大大小是 2。
5. 数组 a 中有 5 个元素，其中奇数有 5, 3, 1 三个，因此执行了 3 次 s.push() 入栈操作。

三、完善程序题（共 2 题，每题 15 分；每题 5 空、每空 3 分，共 30 分）

19. （本题 15 分） 下面程序的功能是实现一个简单的快速傅里叶变换（FFT）算法，用于计算多项式在单位根处的值。程序读入一个整数 n（n 是 2 的幂），然后读入 n 个复数（实部和虚部），表示多项式的系数。要求在 BLANK1~BLANK5 处填入正确的代码，使得程序能够正确输出 FFT 的结果。复数使用 `complex<double>` 表示。

```
#include <iostream>
#include <vector>
#include <complex>
#include <cmath>
using namespace std;

const double PI = acos(-1.0);

void fft(vector<complex<double>>& a, bool invert) {
    int n = a.size();
    if (n == 1) return;

    vector<complex<double>> a0(n / 2), a1(n / 2);
    for (int i = 0; i < n / 2; ++i) {
        ____(2)____
        ____(3)____
    }

    ____(4)____
    fft(a1, invert);

    double ang = 2 * PI / n * (invert ? -1 : 1);
    complex<double> w(1), wn(cos(ang), sin(ang));

    for (int i = 0; i < n / 2; ++i) {
        a[i] = a0[i] + w * a1[i];
        a[i + n / 2] = a0[i] - w * a1[i];
        if (invert) {
            a[i] /= 2;
            a[i + n / 2] /= 2;
        }
        ____(5)____
    }
}
```

```
int main() {
    int n;
    cin >> n;
    vector<complex<double>> a(n);
    for (int i = 0; i < n; ++i) {
        double real, imag;
        cin >> real >> imag;
        a[i] = {real, imag};
    }

    // ____(1)____: 调用 fft 函数进行变换
    ____(1)____;

    // 输出结果
    for (int i = 0; i < n; ++i) {
        cout << a[i].real() << " " << a[i].imag() << endl;
    }
    return 0;
}
```

样例输入：

```
4
1 0
2 0
3 0
4 0
```

样例输出：

```
10 0
-2 -2
-2 0
-2 2
```

空 1 （3 分）

A. fft(a, false);

B. fft(a, true);

C. fft(a, 0);

D. fft(a, 1);

答案： A

空 2 （3 分）

A. a0[i] = a[2 * i];

B. a0[i] = a[2 * i + 1];

C. a0[i] = a[i];

D. a0[i] = a[n - i - 1];

答案： A

空 3 （3 分）

A. a1[i] = a[2 * i];

B. a1[i] = a[2 * i + 1];

C. a1[i] = a[i];

D. a1[i] = a[n - i - 1];

答案： B

空 4 （3 分）

A. fft(a0, invert);

B. fft(a1, invert);

C. fft(a, !invert);

D. fft(a0, !invert);

> **答案：** A

空 5 （3 分）

A. w *= wn;

B. w /= wn;

C. w += wn;

D. w -= wn;

> **答案：** A

> **整题解析：**
> BLANK1 调用 fft 进行正变换，传入 false 或 true 分别表示正逆变换。根据需求，这里是正变换，因此选 A. fft(a, false);。
> BLANK2 将偶数下标的系数放入 a0 数组，正确选项为 A. a0[i] = a[2 * i];。
> BLANK3 将奇数下标的系数放入 a1 数组，正确选项为 B. a1[i] = a[2 * i + 1];。
> BLANK4 递归调用对 a0 进行 FFT，应传递相同的 invert 标志，正确选项为 A. fft(a0, invert);。
> BLANK5 在循环中更新旋转因子 w，需要乘以 wn 以旋转到下一个单位根，正确选项为 A. w *= wn;。
> 整体程序完成 FFT 正变换，示例输入 4 个系数 (1,0),(2,0),(3,0),(4,0) 输出与预期一致。

20. （本题 15 分） 给定B进制的正整数N，判断其是否为回文数（即正读反读相同）。下面程序利用双向链表存储N的每一位，通过首尾比较实现判断。 请在空白处填入正确代码。

```
#include <iostream>
#include <cstring>
using namespace std;
const int MAXN = 100;
struct Node {
    int digit;
    Node *prev, *next;
} *head, *tail;
int main() {
    int B;
    char N[MAXN];
    cin >> B >> N;
    head = tail = NULL;
    int len = strlen(N);
    for (int i = 0; i < len; i++) {
        int d = (N[i] >= '0' && N[i] <= '9') ? N[i] - '0' : N[i] - 'A' + 10;
        Node *p = new Node{d, NULL, NULL};
        if (head == NULL) {
            head = tail = p;
        } else {
            tail->next = p;
            ____(1)____;
            tail = p;
        }
    }
    ____(2)____;
    bool flag = true;
    while (____(3)____) {
```

```
        if (left->digit != right->digit) {
            flag = false;
            break;
        }
        ____(4)____;
        ____(5)____;
    }
    cout << (flag ? "Yes" : "No") << endl;
    return 0;
}
```

样例输入：

```
10 12321
16 A1A
```

样例输出：

```
Yes

Yes
```

空 1 （3 分）

A. p->prev = tail;

B. tail->prev = p;

C. p->next = tail;

D. head->prev = p;

答案： A

空 2 （3 分）

A. Node *left = head, *right = head;

B. Node *left = tail, *right = head;

C. Node *left = head, *right = tail;

D. Node *left = tail, *right = tail;

答案： C

空 3 （3 分）

A. left != right && left->prev != right

B. left != right && left->next != right

C. left != right || left->next != right

D. left != right

答案： B

空 4 （3 分）

A. left = left->prev;

B. left = left->next;

C. right = right->next;

D. left = right->prev;

答案： B

空 5 （3 分）

A. right = right->prev;

B. right = right->next;

C. left = left->next;

D. right = left->next;

答案： A

整题解析：

1. 构建双向链表时，新节点的prev应指向当前尾节点tail，故A正确。若选B则tail->prev指向p，但此时p的prev未正确设置，且链表方向混乱；C使p->next指向tail，非尾插法；D仅修改head->prev，未建立正确的双向链接。
2. 从两端开始比较，left指向头head，right指向尾tail。A、D都使两指针同侧，无法比较；B顺序相反，但影响后续移动。
3. 循环条件需保证left和right未相遇或交叉。偶数长度时相遇时left->next == right；奇数长度时相遇时left == right。双重条件用&&保证两指针内缩过程中不越界。若只判断left!=right，则奇数长度最后会left==right但仍需比较该位（实际上left==right时该位不用比较，但循环终止在相遇前）。然而代码在循环内比较digit，若相遇时left==right，此时不应再进入循环比较，但循环结束后flag仍为true。但这样会在偶数长度时，left和right交叉后无效。更精确的条件是left != right，且当left和right相邻时终止（left->next == right）。但选项A：left != right && left->prev != right，初始left->prev为NULL，不适合。正确条件是left != right && left->next != right，对应选项B。当链表长度为1时不进入循环，长度为2时进入循环比较一次后left右移、right左移，此时left==right不满足left!=right退出，正确。请注意，fullCode中已更新为left != right && left->next != right。
4. left向右移动，应更新left = left->next。A向左；C错误；D含义错误。
5. right向左移动，应更新right = right->prev。B向右；C重复left移动；D错误。